

Sessional Project Report

Designing a 22-bit Mini Computer



Course No. CSE - 2114

Name: Khalid Ahammed Uzzal

Roll: 2207035

Group No. A2

Year: 2nd

Semester: 1st

Date of Performance: 29/01/2025

Date of Submission: 12/02/2025

Abstract

This report presents the design and implementation details of a 22-bit Logisim-based mini-computer. The computer features a 5-bit Program Counter (PC), 5-bit Memory Address Register (MAR), 4-bit Instruction Register (IR), and 32-word RAM. The instruction set includes AND, ADD, STO (STORE), ISZ, BSB, JMP, LOAD, HALT, CMP, JZ, JN, JC, and JO. This report provides an in-depth explanation of the architecture, instruction formats, execution processes, and example implementations.

Objectives

- To develop a thorough understanding of essential CPU components, the memory interface, and the overall operational flow of a processor.
- To explore the concept of instruction sets and analyze their impact on CPU architecture and performance.
- To efficiently encode instructions within a constrained bit-length using various encoding schemes and methodologies.
- To enhance problem-solving skills by debugging and optimizing simple CPU simulations and more complex digital systems.

Introduction

This project involves designing and implementing a simplified 22-bit CPU architecture that serves as a foundational model for understanding essential computing principles. The system follows a hardwired control mechanism to execute a limited but functional set of instructions, offering insights into basic processor operations and instruction execution.

While designed for efficiency, certain bit allocations remain unused, presenting opportunities for future enhancements. This architecture provides a structured approach to learning core principles of computer design.

The computer processes machine code instructions using a structured 6-digit hexadecimal format. Each instruction represents 4-bit binary sequences total of 22 bits,

where the first two bits remain unused, the following 4 bits define the opcode, and the last 5 bits specify the memory address, with other bits left unutilized.

System Architecture

This project involves designing and implementing a simplified 22-bit CPU architecture that serves as a foundational model for understanding essential computing principles. The system follows a hardwired control mechanism to execute a limited but functional set of instructions, offering insights into basic processor operations and instruction execution.

CPU Architecture

- **RAM Word Size:** 32-bit
- **Data Bus:** 22-bit
- **Address Bus:** 5-bit, supporting a 32-word addressable memory

Registers

- **Program Counter (PC):** 5-bit
- **Memory Buffer Register (MBR):** 22-bit
- **Memory Address Register (MAR):** 5-bit
- **Instruction Register (IR):** 4-bit
- **CPL:** 22-bit (4 bit, 2 bit and 1 bit unit)
- **Accumulator:** 22-bit

Arithmetic Logic Unit (ALU) - 22 bit

- Supports operations: ADD, SUB, AND
- Extended operations: INC, JMP, CMP

Control Unit

- Manages instruction execution and coordinates data flow within the CPU

Memory

- **32-word RAM:** Stores program instructions and data

Instruction Set

The instruction set consists of 22-bit.

2 1	2 0	1 9	16	15	5	4	0
		Opcode		Not used			Address
0	0	XXXX		XXX...XX			XXXXX

The bits that are given 20,21 and 5-15 are the unused bits. These are kept unused to read/write the instructions only by seeing the hex values.

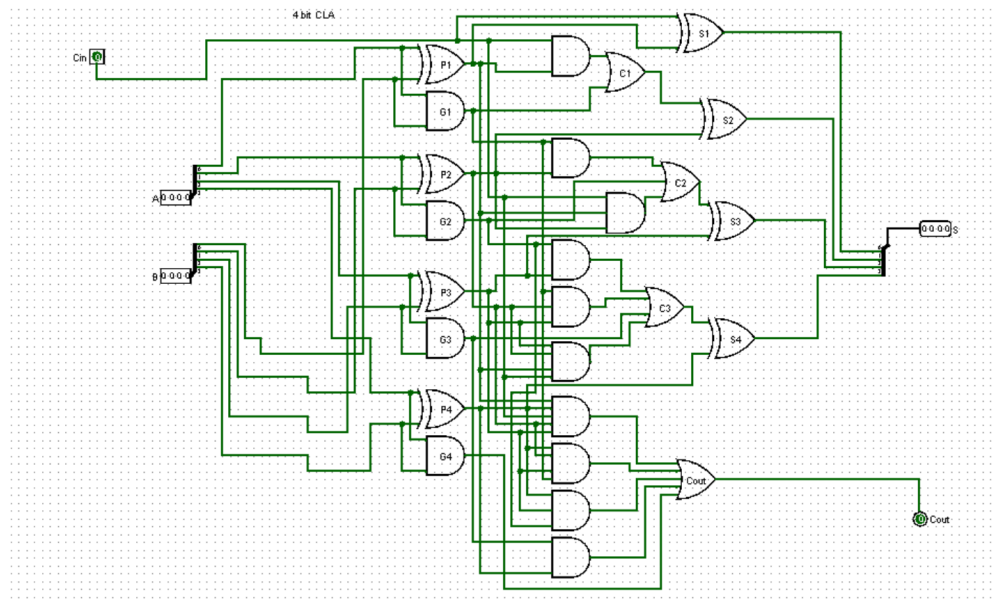
Opcode is of 4 bits, so there are $2^4 = 16$ instructions.

Opcode	Hex value	Mnemonic	Definition
0000	0	AND	$A \leftarrow A \& M$
0001	1	ADD	$A \leftarrow A + M$
0010	2	STO	$M \leftarrow A$
0011	3	ISZ	$M \leftarrow M+1$ ($M+1=0 ? PC \leftarrow PC+1$)
0100	4	BSB	$M \leftarrow PC+5000, PC \leftarrow M+1$
0101	5	JMP	$PC \leftarrow M$
0110	6	LOAD	$A \leftarrow M$
0111	7	HALT	Clock = 0
1000	8	CMP	$A \leftarrow A - M$, STO Flag register (4bit)
1001	9	JZ	isZero=1, $M \leftarrow PC+5000, PC \leftarrow M+1$
1010	A	JN	isNeg=1, $M \leftarrow PC+5000, PC \leftarrow M+1$
1011	B	JC	isCarry=1, $M \leftarrow PC+5000, PC \leftarrow M+1$
1100	C	JO	isOverflow, $M \leftarrow PC+5000, PC \leftarrow M+1$
1101	D	reserved	N/A
1110	E	reserved	N/A
1111	F	reserved	N/A

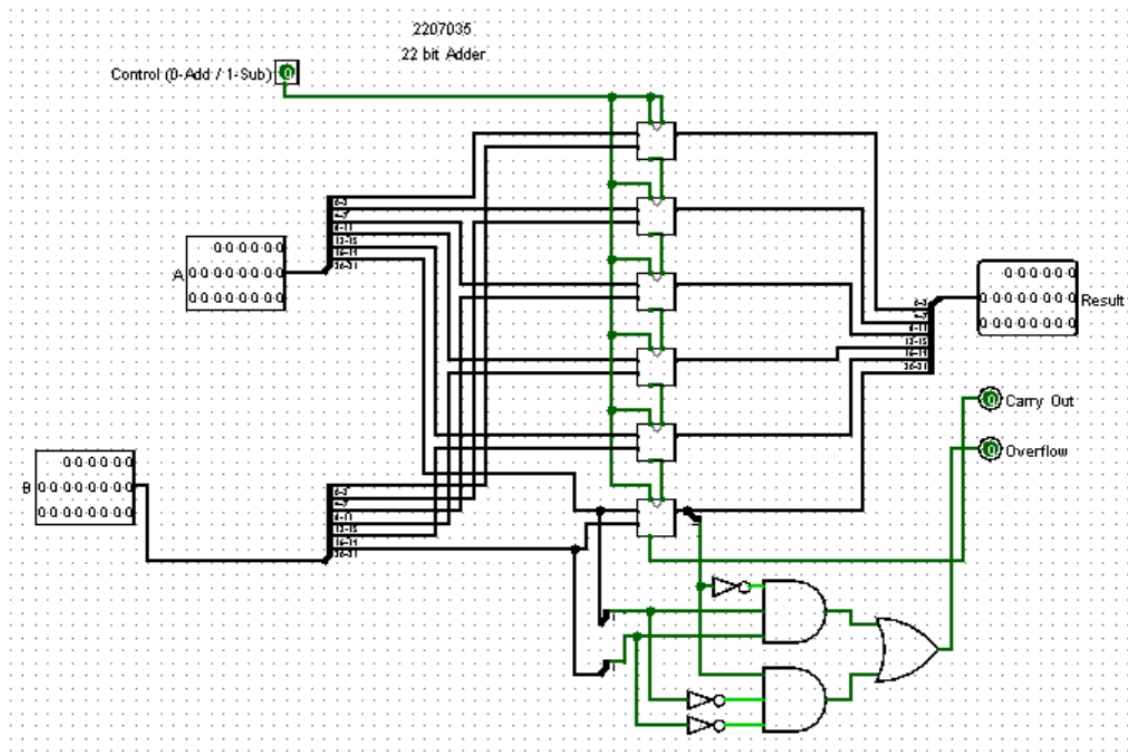
CPU Components

The key components involved in executing the instructions are represented by logic circuits below.

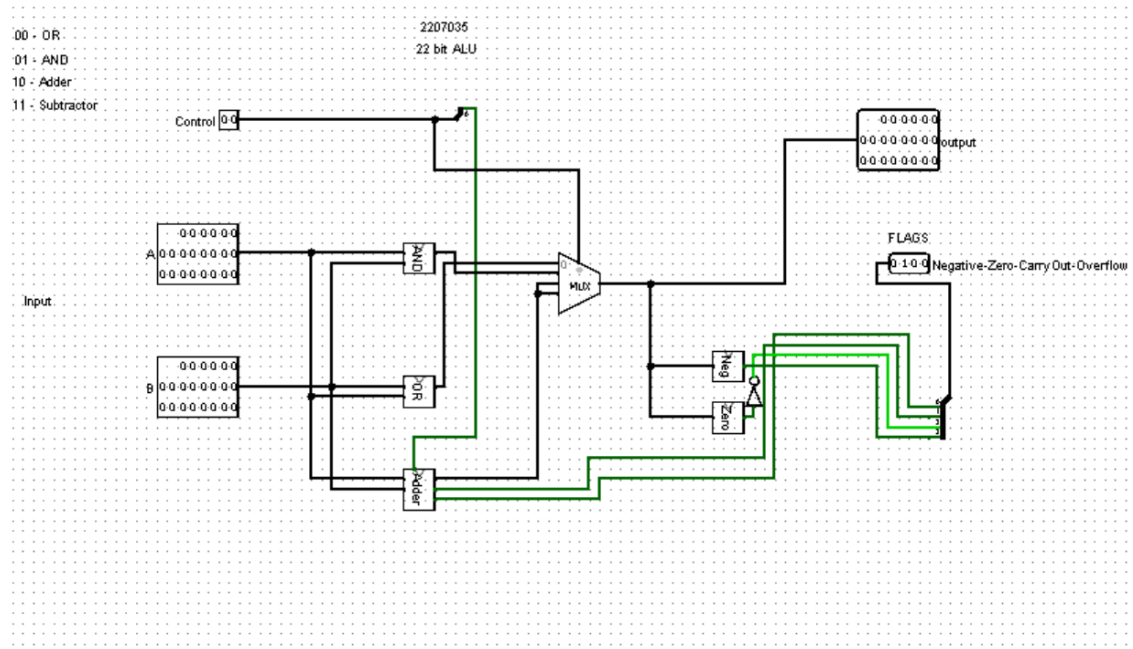
CLA



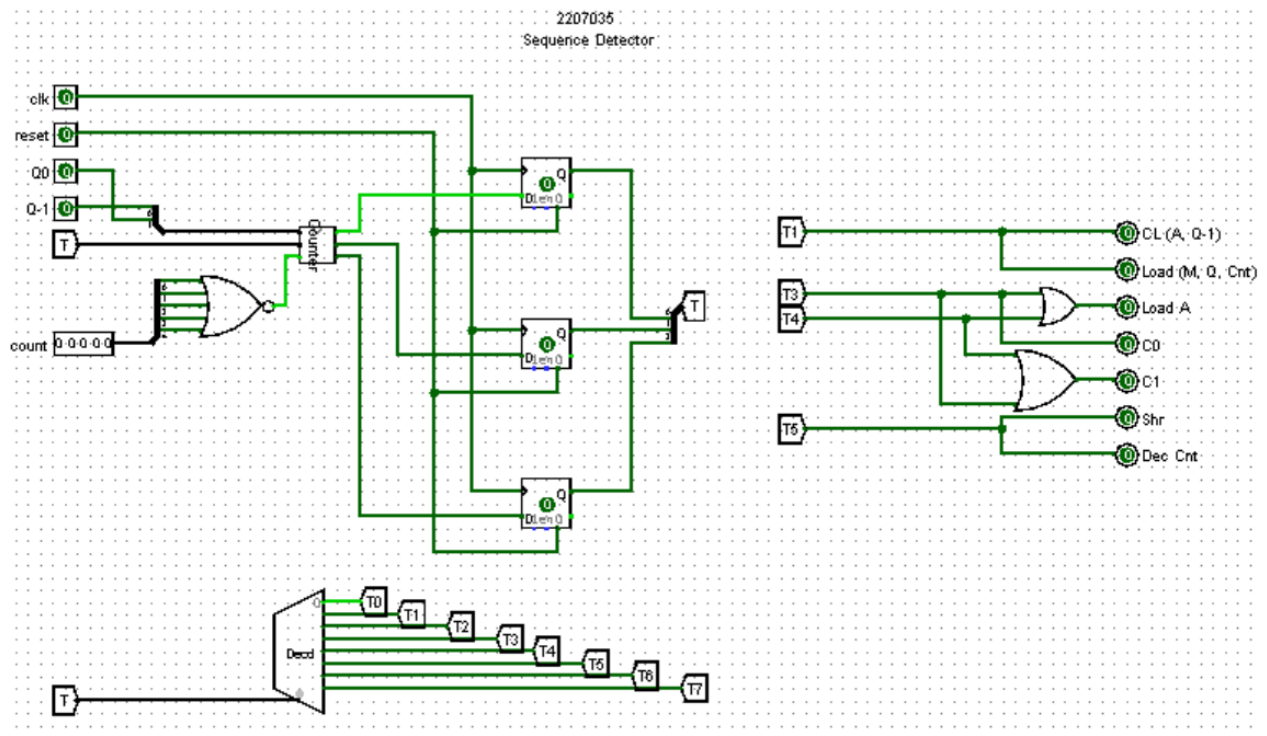
22 bit Adder



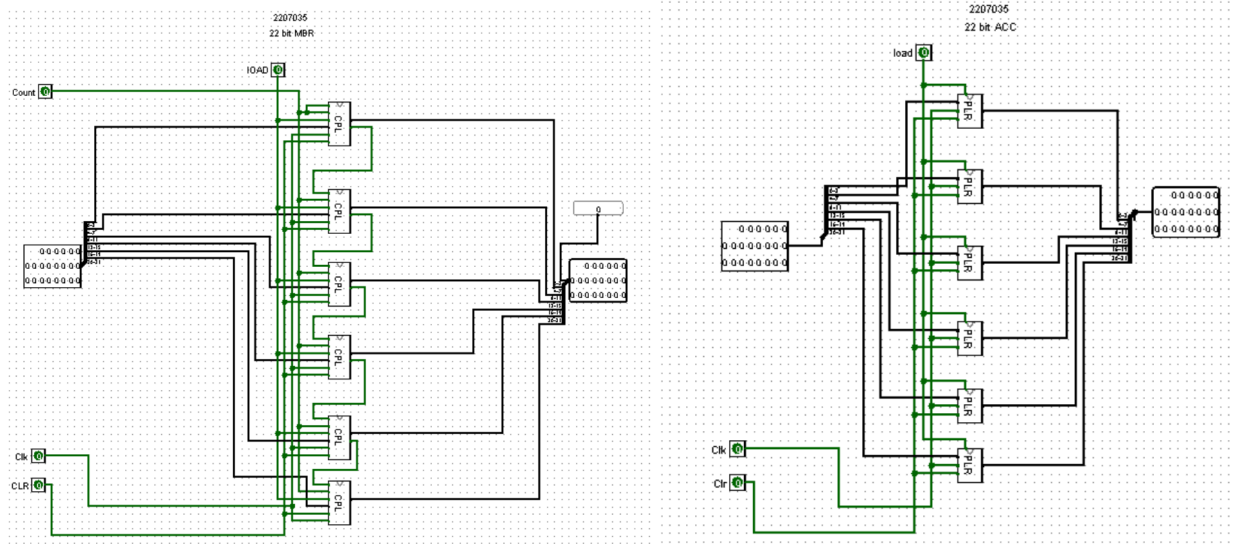
22 bit ALU



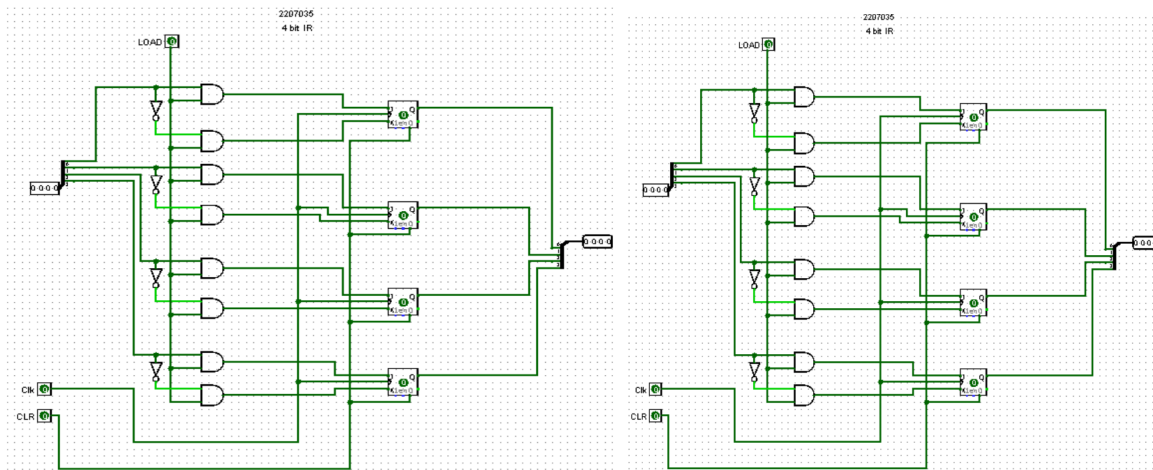
Sequence Detector



22 bit MBR & 22 bit Accumulator



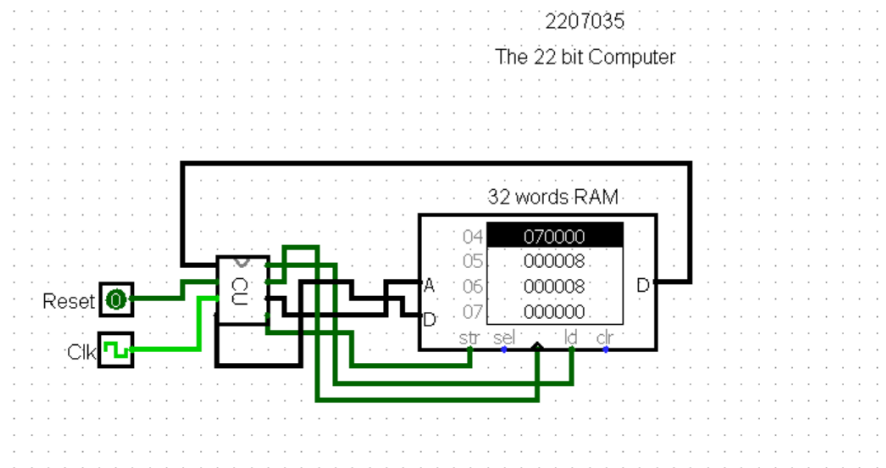
5 bit PC & 4 bit IR



Control Unit

The Control Unit implements the logics behind the instructions in hardware level using the components. It consists of lots of Decoder and tunneling manipulation across different Components, Circuits, Counters and Registers.

22 bit main Computer



Example Instruction Running

In this computer, we the following logic can be implemented the following way.

Logic:

a=8, b=8

Cmp a,b

Store m[loc0], a

If a==b: Jmp l1

Halt

l1:

a=a+b

Store m[loc1], a

Ret

Machine Code (in Hex):

v2.0 raw

060005 080006 020008 0b000b 070000 000008 000004 4*0 060005
010006 020012 050004

Discussion

The design and implementation of the 22-bit mini-computer provided valuable insights into computer architecture, particularly in instruction execution, memory management, and control unit design. The Logisim-based simulation allowed us to visualize the execution of each instruction and understand how different components interact within the CPU.

One of the key challenges encountered was optimizing the bit allocation to ensure efficient instruction encoding. While the design included unused bits, they provide potential for future expansion. The limited memory space of 32 words also posed constraints, but it effectively demonstrated the principles of addressable memory and data storage.

The hardwired control unit proved to be an effective approach for managing instruction execution, though a microprogrammed control unit could offer more flexibility. The inclusion of arithmetic and logical operations, along with jump and conditional instructions, enabled a robust yet simple computing model suitable for fundamental educational purposes.

Through debugging and optimization, we gained hands-on experience in troubleshooting digital logic circuits, improving our problem-solving skills. The project also reinforced theoretical concepts such as register operations, opcode encoding, and data flow within a CPU.

Conclusion

The development of a 22-bit mini-computer using Logisim successfully demonstrated key principles of computer architecture. By implementing a well-structured instruction set and optimizing memory usage, we created a functional and efficient model that executes fundamental operations.

While the project was constrained by limited memory and unused bit allocations, these limitations highlight opportunities for future improvements, such as expanding memory capacity and refining instruction encoding techniques. The

project also showcased the importance of debugging and optimizing digital circuits, emphasizing the necessity of precision in hardware design.

Overall, this project provided a comprehensive understanding of CPU architecture and its essential components, laying a strong foundation for more advanced studies in computer engineering.

References

1. Digital Logic and Computer Design. M. Morris Mano
2. Computer Architecture & Organization by W. Stallings
3. Logisim Documentation. Retrieved from
<https://cburch.com/logisim/docs.html>